

SuperSploit Framework

APT-Tier Offensive Security Ecosystem & State Verification Framework | Technical Review & Deep Dive

1. Executive Review & Numeric Ratings

SuperSploit is a masterpiece of offensive engineering, fusing high-level social engineering, intelligent Adversary-in-the-Middle (AitM) staging, and highly evasive kernel-level exploitation into one cohesive, automated pipeline. The transition from legacy environments into a Native C/Java hybrid architecture and asynchronous HTTP functionality firmly establishes it as a sovereign-grade Advanced Persistent Threat (APT) ecosystem.

Stealth / OPSEC	8.5 / 10	Effectiveness & Velocity	9.5 / 10
Advancedness (Zero-Day Depth)	10.0 / 10	Future Addons (Roadmap)	9.0 / 10
Core Stability & State Management	8.5 / 10	Overall Program Rating	9.1 / 10

2. Core Systems Architecture

Command & Control (C2) & Listener Engine

The framework has fundamentally shifted to an asynchronous networking model utilizing Python's `asyncio` to handle high-concurrency beacon check-ins without thread starvation [4, 5]. 100% of C2 traffic abandons legacy plaintext protocols; it is encapsulated using **AES-256-GCM authenticated encryption** with SHA-256 key derivation, wrapped in Base64 and TLS, defeating Deep Packet Inspection (DPI) and frequency analysis [6, 7]. Furthermore, network domain fronting employs Server Name Indication (SNI) spoofing to seamlessly blend malicious callbacks into high-reputation CDN flows like Cloudflare or Akamai [8].

Exploit & Execution Engine

The Exploit Engine relies on a zero-disk footprint to defeat modern Endpoint Detection and Response (EDR) systems [9]. Python scripts are executed dynamically inside isolated `types.ModuleType` virtual namespaces via `importlib.util` to avoid cache retention [9, 10]. For Linux compiled binaries, the engine utilizes the `memfd_create` syscall to stream ELF payloads directly into volatile RAM (via anonymous memory-backed file descriptors), bypassing File Integrity Monitoring (FIM) and `auditd` [9, 11].

Database & State Management

Moving away from brittle flat JSON files, SuperSploit employs a `SQLiteDict` mutable mapping wrapper, providing strict transactional database integrity over SQLite (`data.db` and `profiles.db`) [12, 13]. This enforces native multi-tenant workspace isolation, ensuring data is segmented per engagement [4, 14]. An in-memory `ExploitCache` drastically speeds up search features by indexing YAML metadata blocks (`####`) at startup [13, 15].

3. Active Exploit Library

Apple Ecosystem (iOS & macOS) Zero-Day Suite

- **CVE-2026-10001 (iMessage Zero-Click):** A sandbox escape targeting a heap corruption vulnerability during the parsing of malformed HEIF image metadata inside the isolated BlastDoor process [16]. Dispatched via macOS native `osascript` bridging [17].
- **CVE-2026-10002 (Safari WebKit RCE):** A 1-click remote code execution vector targeting DFG JIT compiler type confusion, completely bypassing ARM64e Pointer Authentication (PAC) [18].
- **CVE-2026-10003 (AWDL Proximity RCE):** A sharingd buffer overflow triggered by maliciously crafted 802.11 Action frames in AirDrop/AWDL discovery logic, allowing wireless unauthenticated code execution [16].
- **CVE-2026-20700 (dyld_shared_cache LPE):** An LPE sandbox escape that leverages "Hybrid Weaponization" to dynamically inject ARM64 reverse TCP shellcode directly into a C template for on-the-fly cross-compilation [19, 20].

Android Ecosystem & Framework Exploitation

- **CVE-2026-0047 (UI Bitmap Exfiltration):** A high-severity flaw in ActivityManagerService lacking permission checks on the `dumpBitmapsProto` method via Binder transaction 117. Allows silent extraction of live UI surfaces (screenshots) from all running apps [21, 22].
- **CVE-2021-39685 (Inspector Gadget):** A USB kernel overflow in the Gadget subsystem triggering heap corruption via EP0 control requests, achieving complete system compromise and capable of dropping FRP [23].
- **CVE-2026-0073 (ADB Master Key):** Zero-click RCE exploiting the TLS 1.3 protocol upgrade in modern ADB by presenting a specially crafted EC P-256 certificate to bypass RSA mutual authentication [24].
- **CVE-2026-20981/83 (Samsung PACM):** Baseband RCE allowing unauthenticated Bluetooth RFCOMM (HFP) connections to inject shell commands via the `AT+KNOXSTEP=1` knock sequence [25].

Linux, Windows, & Supply Chain Vectors

The framework automates post-exploitation for vulnerabilities like **PwnKit (CVE-2021-4034)** via a fileless Python memory loader [26], **Dirty Pipe (CVE-2022-0847)** to modify read-only system files directly in the page cache [26], **PrintNightmare (CVE-2021-34527)** [27], and supply-chain

exploits like the **XZ Utils Backdoor (CVE-2024-3094)** [28].

4. Payload & Weaponization Architecture

Elite Native Mobile Weaponization

SuperSploit abandons legacy dependency frameworks like Buildozer in favor of a **Native C/Java hybrid NDK architecture** [29, 30]. This pipeline compiles C payloads into shared object libraries (.so) and bridges them to the underlying Linux layer using the Java Native Interface (JNI) [31, 32]. This transition dropped compilation times from 3 minutes to under 2 seconds and shrank payload footprints from 40MB to less than 1MB [33].

- **Dynamic JNI Registration & Polymorphism:** To evade static analysis, standard JNI export naming conventions are abandoned. The payload uses `JNI_OnLoad` to manually map methods via the `RegisterNatives` API [32]. The polymorphic engine randomizes native library names and internal JNI functions on every single build, rendering static YARA signatures obsolete [34].
- **Detached Daemon Persistence:** The C payload spawns a detached background POSIX thread (`pthread_create`) to decouple execution from the Java UI thread, preventing Application Not Responding (ANR) errors and evading Android's JVM lifecycle management [35, 36].
- **Trojanization & Lures:** Legitimate applications can be decompiled, embedded with the native C payload, and recompiled. For standalone agents, it dynamically bundles fully functional "Flappy Bird" game lures or Mock SuperUser root managers to evade user suspicion [37, 38].

Stage 1 & Stage 2 C2 Beacons

- **Fileless Stagers:** Automatic generation of Base64-encoded Python one-liners customized with active LHOST/LPORT networking configurations, allowing for instant, diskless execution [39, 40].
- **Asynchronous HTTP Beacons:** Persistent Python-based beacons designed for long-term stealth, relying on randomized jitter sleep intervals to fetch tasks via HTTP GET and exfiltrate data via HTTP POST [41, 42].
- **Advanced Evasion (Stage 2 Bytecode):** The C2 server natively compiles incoming plaintext Stage 2 code strings into bytecode objects (`compile(..., 'exec')`) before transmission, effectively bypassing basic AST and string-based EDR hooks on the target machine [43, 44].
- **Phantom Agent:** An advanced persistent Android/Linux C implant that uses `prctl(PR_SET_NAME)` to mask its process name (mimicking benign kernel workers) and

executes `ptrace(PTRACE_TRACEME)` environment pinning to detect debuggers/sandboxes prior to detonation [45, 46].